# 🚀 BOOKYOURSERVICE - ULTIMATE PRODUCTION-GRADE MARKETPLACE DEVELOPMENT PROMPT

## 📋 EXECUTIVE SUMMARY

Build a **world-class, enterprise-ready service marketplace platform** named **BookYourService** (bookyourservice.co.in) connecting clients with verified service providers globally. This is a **ZERO-COMPROMISE, production-first** development mandate with no mock data, no test shortcuts, and no assumptions.

---

## 🎯 PROJECT SPECIFICATIONS

### Project Identity

- **Name:** BookYourService
- **Domain:** bookyourservice.co.in
- **Scope:** Global marketplace (worldwide deployment)
- **Launch Standard:** Bank-grade security, 99.9% uptime, scalable to 100K+ concurrent users

### Core Business Model

- **Actors:** Clients, Service Providers, Platform Admin
- **Transaction:** Location-based service booking with verified providers
- **Revenue:** Platform fee (minimum ₹5 per booking) + 50+ passive income streams
- **KYC:** Mandatory provider verification before service activation
- **Payment:** Integrated gateway (Razorpay/Stripe) with escrow mechanism

---

## 🏗️ COMPLETE TECHNICAL ARCHITECTURE

### Technology Stack (Production-Grade)

#### Frontend

- **Framework:** React 18.x + TypeScript
- **Build:** Vite 5.x (optimized production builds)
- **State Management:** Context API + React Query
- **UI Components:** shadcn/ui + Tailwind CSS 3.x
- **Maps Integration:** Google Maps API (location, geocoding, distance matrix)
- **Forms:** React Hook Form + Zod validation

- **PWA:** Service Workers, offline support, app-like experience

**Backend**

- **Runtime:** Node.js 20.x LTS
- **Framework:** Express.js 4.x with TypeScript
- **ORM:** Prisma 5.x (type-safe database client)
- **Authentication:** JWT (access + refresh tokens) + HttpOnly cookies
- **API Documentation:** Swagger/OpenAPI 3.0
- **Caching:** Redis 7.x (session store, rate limiting)
- **Queue:** BullMQ (async jobs, notifications, email)

**Database**

- **Primary:** PostgreSQL 16.x (ACID compliance, JSON support)
- **Migration:** Prisma Migrate (version-controlled schema changes)
- **Backup:** Automated daily backups with 30-day retention
- **Replication:** Master-slave setup for read scalability

**Infrastructure**

- **Containerization:** Docker + Docker Compose
- **Orchestration:** Kubernetes (production scaling)
- **CI/CD:** GitHub Actions (automated testing, building, deployment)
- **Hosting:** AWS/GCP/Azure (multi-region deployment)
- **CDN:** CloudFront/Cloudflare (static assets, DDoS protection)
- **SSL:** Let's Encrypt (automated certificate renewal)

**Monitoring & Observability**

- **Logging:** Winston + ELK Stack (Elasticsearch, Logstash, Kibana)
- **Monitoring:** Prometheus + Grafana
- **Error Tracking:** Sentry
- **Uptime:** UptimeRobot + PagerDuty alerts
- **Performance:** New Relic / DataDog APM

**Security**

- **WAF:** Web Application Firewall (AWS WAF/Cloudflare)
- **Rate Limiting:** Express-rate-limit + Redis

- **OWASP:** Top 10 compliance (SQL injection, XSS, CSRF protection)
- **Data Encryption:** AES-256 at rest, TLS 1.3 in transit
- **PCI DSS:** Payment gateway compliance
- **GDPR:** Data privacy, right to erasure

---

## 📊 COMPLETE DATABASE SCHEMA (PRODUCTION-READY)

### Core Tables

#### 1. Users Table

sql

```sql
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(20) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    role_id INTEGER NOT NULL REFERENCES roles(id),
    status VARCHAR(20) DEFAULT 'PENDING' CHECK (status IN ('ACTIVE', 'BLOCKED', 'PENDING', 'SUSPENDED')),
    email_verified BOOLEAN DEFAULT FALSE,
    phone_verified BOOLEAN DEFAULT FALSE,
    profile_image_url TEXT,
    address JSONB,
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    login_attempts INTEGER DEFAULT 0,
    last_login_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_phone ON users(phone);
CREATE INDEX idx_users_location ON users(latitude, longitude);
CREATE INDEX idx_users_status ON users(status);
```

#### 2. Roles Table

sql

```
CREATE TABLE roles (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL CHECK (name IN ('CLIENT', 'PROVIDER', 'ADMIN')),
    description TEXT,
    permissions JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO roles (name, description) VALUES
    ('CLIENT', 'Service consumer'),
    ('PROVIDER', 'Service provider'),
    ('ADMIN', 'Platform administrator');
```

### 3. Provider KYC Table

sql

```
CREATE TABLE provider_kyc (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    document_type VARCHAR(50) NOT NULL CHECK (document_type IN ('AADHAAR', 'PAN', 'DRIVING_LICENSE', 'P
    document_number VARCHAR(100) NOT NULL,
    document_front_url TEXT NOT NULL,
    document_back_url TEXT,
    selfie_url TEXT NOT NULL,
    verification_status VARCHAR(20) DEFAULT 'PENDING' CHECK (verification_status IN ('PENDING', 'APPROVED', 'R
    verified_by UUID REFERENCES users(id),
    verified_at TIMESTAMP,
    rejection_reason TEXT,
    expires_at DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(provider_id, document_type)
);

CREATE INDEX idx_kyc_provider ON provider_kyc(provider_id);
CREATE INDEX idx_kyc_status ON provider_kyc(verification_status);
```

### 4. Service Categories (25+ Categories)

sql

```sql
CREATE TABLE service_categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    slug VARCHAR(100) UNIQUE NOT NULL,
    description TEXT,
    icon_url TEXT,
    parent_id INTEGER REFERENCES service_categories(id),
    is_active BOOLEAN DEFAULT TRUE,
    display_order INTEGER,
    seo_title VARCHAR(255),
    seo_description TEXT,
    seo_keywords TEXT[],
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Main Categories (25+)
INSERT INTO service_categories (name, slug, description) VALUES
    ('Home Maintenance & Repairs', 'home-maintenance', 'Essential home repair services'),
    ('Appliances & Electronics', 'appliances-electronics', 'Appliance and gadget repair'),
    ('Outdoor & Property', 'outdoor-property', 'Lawn, cleaning, moving services'),
    ('Beauty & Wellness', 'beauty-wellness', 'At-home salon and spa services'),
    ('Professional Services', 'professional-services', 'Tutoring, events, photography'),
    ('AC & HVAC Services', 'ac-hvac', 'Cooling and heating solutions'),
    ('Plumbing Services', 'plumbing', 'Water and drainage solutions'),
    ('Electrical Services', 'electrical', 'Wiring and electrical repairs'),
    ('Carpentry & Woodwork', 'carpentry', 'Furniture and wood services'),
    ('Painting & Decoration', 'painting', 'Interior and exterior painting'),
    ('Handyman Services', 'handyman', 'General odd jobs'),
    ('Masonry & Tiling', 'masonry', 'Floor and wall tiling'),
    ('Pest Control', 'pest-control', 'Pest elimination services'),
    ('Home Cleaning', 'home-cleaning', 'Deep and regular cleaning'),
    ('Water Tank Cleaning', 'water-tank-cleaning', 'Tank and drainage cleaning'),
    ('Appliance Repair', 'appliance-repair', 'Fridge, washing machine, etc.'),
    ('Gadget Repair', 'gadget-repair', 'Smartphone, laptop, tablet repair'),
    ('Lawn & Gardening', 'lawn-gardening', 'Garden maintenance'),
    ('Exterior Cleaning', 'exterior-cleaning', 'Pressure washing, roof cleaning'),
    ('Moving & Relocation', 'moving-relocation', 'Packing and shifting services'),
    ('Salon for Women', 'salon-women', 'Hair, makeup, beauty services'),
    ('Barber for Men', 'barber-men', 'Haircut, shave, grooming'),
    ('Massage & Spa', 'massage-spa', 'Therapeutic massage services'),
    ('Fitness Training', 'fitness-training', 'Personal training, yoga'),
    ('Home Tutoring', 'home-tutoring', 'Academic and skill tutoring'),
    ('Event Planning', 'event-planning', 'Party and event management'),
    ('Photography & Video', 'photography-video', 'Professional photography'),
    ('Tailoring & Alteration', 'tailoring', 'Clothing repair and custom'),
```

```
    ('Pet Care', 'pet-care', 'Dog walking, grooming, sitting'),
    ('Car Care', 'car-care', 'Car wash and detailing');

CREATE INDEX idx_categories_slug ON service_categories(slug);
CREATE INDEX idx_categories_active ON service_categories(is_active);
```

### 5. Service Subcategories (15+ per category)

```
sql

CREATE TABLE service_subcategories (
    id SERIAL PRIMARY KEY,
    category_id INTEGER NOT NULL REFERENCES service_categories(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    slug VARCHAR(100) NOT NULL,
    description TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    display_order INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(category_id, slug)
);

-- Example: Plumbing subcategories (15+)
INSERT INTO service_subcategories (category_id, name, slug) VALUES
    (7, 'Leak Repair', 'leak-repair'),
    (7, 'Drain Cleaning', 'drain-cleaning'),
    (7, 'Pipe Installation', 'pipe-installation'),
    (7, 'Faucet Repair', 'faucet-repair'),
    (7, 'Toilet Installation', 'toilet-installation'),
    (7, 'Water Heater Repair', 'water-heater-repair'),
    (7, 'Sewage Cleaning', 'sewage-cleaning'),
    (7, 'Shower Tub Repair', 'shower-tub-repair'),
    (7, 'Gas Line Servicing', 'gas-line-servicing'),
    (7, 'Pump Repair', 'pump-repair'),
    (7, 'Bathroom Fitting', 'bathroom-fitting'),
    (7, 'Kitchen Plumbing', 'kitchen-plumbing'),
    (7, 'Water Pressure Issues', 'water-pressure-issues'),
    (7, 'Sump Pump Installation', 'sump-pump-installation'),
    (7, 'Tankless Water Heater', 'tankless-water-heater');

CREATE INDEX idx_subcategories_category ON service_subcategories(category_id);
CREATE INDEX idx_subcategories_slug ON service_subcategories(slug);
```

### 6. Services Table

sql

```sql
CREATE TABLE services (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    category_id INTEGER NOT NULL REFERENCES service_categories(id),
    subcategory_id INTEGER REFERENCES service_subcategories(id),
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    base_price DECIMAL(10, 2) NOT NULL CHECK (base_price > 0),
    price_negotiable BOOLEAN DEFAULT FALSE,
    service_duration_minutes INTEGER,
    service_area_radius_km INTEGER DEFAULT 10,
    latitude DECIMAL(10, 8) NOT NULL,
    longitude DECIMAL(11, 8) NOT NULL,
    address TEXT NOT NULL,
    city VARCHAR(100),
    state VARCHAR(100),
    country VARCHAR(100),
    pincode VARCHAR(20),
    images JSONB,
    availability JSONB,
    is_active BOOLEAN DEFAULT FALSE,
    is_approved BOOLEAN DEFAULT FALSE,
    approval_status VARCHAR(20) DEFAULT 'PENDING' CHECK (approval_status IN ('PENDING', 'APPROVED', 'REJEC
    approved_by UUID REFERENCES users(id),
    approved_at TIMESTAMP,
    rejection_reason TEXT,
    average_rating DECIMAL(3, 2) DEFAULT 0.00,
    total_bookings INTEGER DEFAULT 0,
    total_reviews INTEGER DEFAULT 0,
    seo_title VARCHAR(255),
    seo_description TEXT,
    seo_keywords TEXT[],
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP
);

CREATE INDEX idx_services_provider ON services(provider_id);
CREATE INDEX idx_services_category ON services(category_id);
CREATE INDEX idx_services_location ON services(latitude, longitude);
CREATE INDEX idx_services_active_approved ON services(is_active, is_approved);
CREATE INDEX idx_services_rating ON services(average_rating DESC);
```

### 7. Service Availability Slots

```sql
CREATE TABLE service_availability (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE,
    day_of_week INTEGER NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    is_available BOOLEAN DEFAULT TRUE,
    max_bookings_per_slot INTEGER DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CHECK (end_time > start_time)
);

CREATE INDEX idx_availability_service ON service_availability(service_id);
CREATE INDEX idx_availability_day ON service_availability(day_of_week);
```

### 8. Bookings Table (State Machine)

```sql
```

```sql
CREATE TABLE bookings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_number VARCHAR(50) UNIQUE NOT NULL,
    client_id UUID NOT NULL REFERENCES users(id),
    provider_id UUID NOT NULL REFERENCES users(id),
    service_id UUID NOT NULL REFERENCES services(id),
    status VARCHAR(20) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'ACCEPTED', 'IN_PROGRESS', 'COMPL
    scheduled_date DATE NOT NULL,
    scheduled_time TIME NOT NULL,
    service_address TEXT NOT NULL,
    service_latitude DECIMAL(10, 8),
    service_longitude DECIMAL(11, 8),
    distance_km DECIMAL(8, 2),
    base_price DECIMAL(10, 2) NOT NULL,
    negotiated_price DECIMAL(10, 2),
    final_price DECIMAL(10, 2) NOT NULL,
    platform_fee DECIMAL(10, 2) NOT NULL CHECK (platform_fee >= 5.00),
    provider_earnings DECIMAL(10, 2),
    special_instructions TEXT,
    cancellation_reason TEXT,
    cancelled_by UUID REFERENCES users(id),
    cancelled_at TIMESTAMP,
    completed_at TIMESTAMP,
    payment_status VARCHAR(20) DEFAULT 'PENDING' CHECK (payment_status IN ('PENDING', 'PAID', 'REFUNDED',
    payment_id UUID REFERENCES payments(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_bookings_client ON bookings(client_id);
CREATE INDEX idx_bookings_provider ON bookings(provider_id);
CREATE INDEX idx_bookings_service ON bookings(service_id);
CREATE INDEX idx_bookings_status ON bookings(status);
CREATE INDEX idx_bookings_scheduled ON bookings(scheduled_date, scheduled_time);
CREATE INDEX idx_bookings_number ON bookings(booking_number);
```

### 9. Payments Table

sql

```sql
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID UNIQUE NOT NULL REFERENCES bookings(id) ON DELETE RESTRICT,
    amount DECIMAL(10, 2) NOT NULL,
    currency VARCHAR(10) DEFAULT 'INR',
    payment_method VARCHAR(50) CHECK (payment_method IN ('CARD', 'UPI', 'WALLET', 'NET_BANKING')),
    gateway VARCHAR(50) CHECK (gateway IN ('RAZORPAY', 'STRIPE', 'PAYPAL')),
    gateway_order_id VARCHAR(255),
    gateway_payment_id VARCHAR(255),
    gateway_signature VARCHAR(500),
    status VARCHAR(20) DEFAULT 'CREATED' CHECK (status IN ('CREATED', 'PROCESSING', 'SUCCESS', 'FAILED', 'I
    verified BOOLEAN DEFAULT FALSE,
    refund_amount DECIMAL(10, 2),
    refund_reason TEXT,
    refunded_at TIMESTAMP,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_payments_booking ON payments(booking_id);
CREATE INDEX idx_payments_status ON payments(status);
CREATE INDEX idx_payments_gateway_order ON payments(gateway_order_id);
```

### 10. Reviews & Ratings

sql

```sql
CREATE TABLE reviews (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID UNIQUE NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    reviewer_id UUID NOT NULL REFERENCES users(id),
    reviewed_id UUID NOT NULL REFERENCES users(id),
    service_id UUID NOT NULL REFERENCES services(id),
    rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment TEXT,
    images JSONB,
    is_verified BOOLEAN DEFAULT FALSE,
    is_flagged BOOLEAN DEFAULT FALSE,
    flag_reason TEXT,
    admin_response TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_reviews_booking ON reviews(booking_id);
CREATE INDEX idx_reviews_service ON reviews(service_id);
CREATE INDEX idx_reviews_provider ON reviews(reviewed_id);
```

### 11. Price Negotiations

```sql
CREATE TABLE negotiations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    proposed_by UUID NOT NULL REFERENCES users(id),
    proposed_price DECIMAL(10, 2) NOT NULL,
    message TEXT,
    status VARCHAR(20) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'ACCEPTED', 'REJECTED', 'COUNTER')
    responded_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_negotiations_booking ON negotiations(booking_id);
```

### 12. Disputes

```sql
```

```sql
CREATE TABLE disputes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id),
    raised_by UUID NOT NULL REFERENCES users(id),
    dispute_type VARCHAR(50) CHECK (dispute_type IN ('PAYMENT', 'SERVICE_QUALITY', 'NO_SHOW', 'CANCELLA
    description TEXT NOT NULL,
    evidence JSONB,
    status VARCHAR(20) DEFAULT 'OPEN' CHECK (status IN ('OPEN', 'UNDER_REVIEW', 'RESOLVED', 'CLOSED')),
    assigned_to UUID REFERENCES users(id),
    resolution TEXT,
    resolved_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_disputes_booking ON disputes(booking_id);
CREATE INDEX idx_disputes_status ON disputes(status);
```

### 13. Admin Activity Logs

sql

```sql
CREATE TABLE admin_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    admin_id UUID NOT NULL REFERENCES users(id),
    action VARCHAR(100) NOT NULL,
    target_type VARCHAR(50),
    target_id UUID,
    details JSONB,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_admin_logs_admin ON admin_logs(admin_id);
CREATE INDEX idx_admin_logs_action ON admin_logs(action);
CREATE INDEX idx_admin_logs_created ON admin_logs(created_at DESC);
```

### 14. Notifications

sql

```sql
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type VARCHAR(50) NOT NULL,
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    action_url TEXT,
    is_read BOOLEAN DEFAULT FALSE,
    read_at TIMESTAMP,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notifications_user ON notifications(user_id);
CREATE INDEX idx_notifications_read ON notifications(is_read);
```

### 15. FAQ

```sql
CREATE TABLE faqs (
    id SERIAL PRIMARY KEY,
    category VARCHAR(100),
    question TEXT NOT NULL,
    answer TEXT NOT NULL,
    display_order INTEGER,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_faqs_category ON faqs(category);
CREATE INDEX idx_faqs_active ON faqs(is_active);
```

### 16. Legal Pages

```sql
```

```sql
CREATE TABLE legal_pages (
    id SERIAL PRIMARY KEY,
    page_type VARCHAR(50) UNIQUE NOT NULL CHECK (page_type IN ('TERMS', 'PRIVACY', 'REFUND', 'COOKIES')
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    version VARCHAR(20),
    effective_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

### 17. SEO Metadata

```sql
CREATE TABLE seo_metadata (
    id SERIAL PRIMARY KEY,
    page_type VARCHAR(50) NOT NULL,
    page_id VARCHAR(255),
    title VARCHAR(255),
    description TEXT,
    keywords TEXT[],
    canonical_url TEXT,
    og_image TEXT,
    schema_markup JSONB,
    indexed BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_seo_page ON seo_metadata(page_type, page_id);
```

### 18. Platform Revenue Streams (50+)

```sql
```

```
CREATE TABLE revenue_streams (
    id SERIAL PRIMARY KEY,
    stream_type VARCHAR(100) NOT NULL,
    description TEXT,
    revenue_model VARCHAR(50) CHECK (revenue_model IN ('COMMISSION', 'SUBSCRIPTION', 'ADVERTISING', 'FE
    status VARCHAR(20) DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'INACTIVE', 'PLANNED')),
    estimated_monthly_revenue DECIMAL(12, 2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 50+ Revenue Streams
INSERT INTO revenue_streams (stream_type, revenue_model, status) VALUES
    ('Platform Booking Fee', 'COMMISSION', 'ACTIVE'),
    ('Provider Subscription', 'SUBSCRIPTION', 'ACTIVE'),
    ('Featured Service Listings', 'FEATURED_LISTING', 'ACTIVE'),
    ('Banner Advertisements', 'ADVERTISING', 'ACTIVE'),
    ('Sponsored Search Results', 'ADVERTISING', 'ACTIVE'),
    ('Premium Provider Badge', 'PREMIUM', 'ACTIVE'),
    ('Referral Commission', 'REFERRAL', 'ACTIVE'),
    ('Client Membership', 'SUBSCRIPTION', 'PLANNED'),
    ('API Access for Enterprises', 'API_ACCESS', 'PLANNED'),
    ('Data Analytics Services', 'DATA_LICENSING', 'PLANNED'),
    ('Insurance Partnership', 'COMMISSION', 'PLANNED'),
    ('Background Verification Fee', 'COMMISSION', 'ACTIVE'),
    ('Express Booking Fee', 'PREMIUM', 'ACTIVE'),
    ('Cancellation Fee', 'COMMISSION', 'ACTIVE'),
    ('Payment Gateway Fee Share', 'COMMISSION', 'ACTIVE'),
    ('Lead Generation Fee', 'COMMISSION', 'PLANNED'),
    ('White Label Solutions', 'SUBSCRIPTION', 'PLANNED'),
    ('Training Program Fee', 'SUBSCRIPTION', 'PLANNED'),
    ('Certification Programs', 'SUBSCRIPTION', 'PLANNED'),
    ('Equipment Rental Commission', 'COMMISSION', 'PLANNED'),
    ('Product Marketplace', 'COMMISSION', 'PLANNED'),
    ('Service Warranty Plans', 'SUBSCRIPTION', 'PLANNED'),
    ('Dispute Resolution Fee', 'COMMISSION', 'ACTIVE'),
    ('Priority Support Subscription', 'SUBSCRIPTION', 'PLANNED'),
    ('Marketing Tools Subscription', 'SUBSCRIPTION', 'PLANNED'),
    ('Business Analytics Dashboard', 'SUBSCRIPTION', 'PLANNED'),
    ('Multi-Provider Packages', 'COMMISSION', 'PLANNED'),
    ('Seasonal Promotions', 'ADVERTISING', 'ACTIVE'),
    ('Partner Integrations', 'API_ACCESS', 'PLANNED'),
    ('Data Insights for Brands', 'DATA_LICENSING', 'PLANNED'),
    ('SMS Notification Service', 'SUBSCRIPTION', 'PLANNED'),
    ('Email Marketing Tools', 'SUBSCRIPTION', 'PLANNED'),
    ('Advanced Booking Analytics', 'PREMIUM', 'PLANNED'),
    ('Customer Retention Tools', 'SUBSCRIPTION', 'PLANNED'),
```

```
('Automated Review Requests', 'PREMIUM', 'PLANNED'),
('Geo-Fencing Advertising', 'ADVERTISING', 'PLANNED'),
('Influencer Partnerships', 'ADVERTISING', 'PLANNED'),
('Corporate Account Management', 'SUBSCRIPTION', 'PLANNED'),
('Bulk Booking Discounts', 'COMMISSION', 'PLANNED'),
('Referral Network Bonuses', 'REFERRAL', 'ACTIVE'),
('Affiliate Marketing Program', 'REFERRAL', 'PLANNED'),
('Franchising Opportunities', 'SUBSCRIPTION', 'PLANNED'),
('International Expansion Fees', 'COMMISSION', 'PLANNED'),
('Currency Conversion Fees', 'COMMISSION', 'PLANNED'),
('Cross-Border Service Fees', 'COMMISSION', 'PLANNED'),
('Premium Customer Support', 'SUBSCRIPTION', 'PLANNED'),
('Virtual Service Offerings', 'COMMISSION', 'PLANNED'),
('On-Demand Training Videos', 'SUBSCRIPTION', 'PLANNED'),
('Provider Onboarding Fee', 'COMMISSION', 'ACTIVE'),
('Advanced Search Filters', 'PREMIUM', 'PLANNED');
```

---

## 🔄 COMPLETE WORKFLOW IMPLEMENTATION

### 1. User Registration & Authentication

#### Client Registration Flow

```
1. User enters email, phone, password
2. Backend validates (Zod schema):
   - Email format (RFC 5322)
   - Phone format (E.164)
   - Password strength (min 8 chars, uppercase, lowercase, digit, special char)
3. Hash password (bcrypt rounds=12)
4. Create user record (status='PENDING', role='CLIENT')
5. Send OTP (Twilio SMS + Email verification link)
6. User verifies email/phone
7. Update user status to 'ACTIVE'
8. Issue JWT tokens (access: 15min, refresh: 7days)
9. Store refresh token in Redis
10. Set HttpOnly cookie
```

#### Provider Registration Flow

1-7. Same as client registration
8. Redirect to KYC verification page
9. Provider uploads:
   - Government ID (Aadhaar/PAN/Passport)
   - Selfie with ID
   - Business license (if applicable)
10. Documents stored in S3 (encrypted)
11. Admin receives KYC verification request
12. Admin reviews documents (approve/reject)
13. If approved: provider_kyc.verification_status = 'APPROVED'
14. If rejected: send rejection email with reason
15. Provider can create services only after KYC approval

## 2. Service Creation & Approval

### Provider Service Listing

1. Provider (KYC approved) creates service
2. Form inputs:
   - Category & Subcategory (dropdown)
   - Title, Description
   - Base Price (min ₹50)
   - Price Negotiable (checkbox)
   - Service Duration
   - Service Area (Google Maps Autocomplete)
   - Coordinates (Google Geocoding API)
   - Availability Slots (day/time picker)
   - Service Images (max 5, S3 upload)
3. Backend validation:
   - Provider KYC verified
   - All required fields present
   - Coordinates within service area
4. Create service record (is_active=false, is_approved=false)
5. Notify admin for approval
6. Admin reviews service
7. Admin approves/rejects:
   - If approved: is_active=true, is_approved=true
   - If rejected: send rejection reason
8. Service goes live, appears in search

## 3. Service Discovery & Booking

### Client Search Flow

1. Client enters location (Google Places Autocomplete)
2. Client selects category/subcategory
3. Backend queries:
   - Find services within radius (PostGIS/ST_Distance)
   - Filter: is_active=true, is_approved=true
   - Sort: distance ASC, rating DESC
4. Return services with:
   - Provider name, rating, reviews
   - Distance from client
   - Base price
   - Availability
5. Client clicks service -> Service Detail Page
6. Client views:
   - Full description, images
   - Provider profile (rating, experience, reviews)
   - Availability calendar
   - Past reviews
7. Client selects date/time slot
8. Client enters service address (Google Maps)
9. Calculate distance (Google Distance Matrix API)
10. Display final price (base + distance surcharge if applicable)
11. If price_negotiable=true, show "Negotiate Price" button

### Price Negotiation Flow (Optional)

1. Client clicks "Negotiate Price"
2. Client proposes price + message
3. Create negotiation record (status='PENDING')
4. Notify provider via push/SMS/email
5. Provider receives negotiation request
6. Provider can:
   - Accept proposed price
   - Reject
   - Counter with new price
7. If accepted: booking created with negotiated_price
8. If rejected/counter: client notified
9. Max 3 negotiation rounds per booking

### Booking Creation Flow

1. Client confirms booking details
2. Backend checks:
   - Service active & approved
   - Provider available for slot
   - No conflicting booking for same slot
3. Start DB transaction:
   - Lock slot (SELECT FOR UPDATE)
   - Create booking (status='PENDING')
   - Calculate platform fee (max(₹5, base_price * 0.05))
   - Create payment order (Razorpay/Stripe)
   - Commit transaction
4. Redirect client to payment gateway
5. Client completes payment
6. Payment gateway webhook received
7. Verify webhook signature (HMAC SHA256)
8. Update payment status='SUCCESS', verified=true
9. Update booking payment_status='PAID'
10. Notify provider (push/SMS/email)
11. Provider accepts/rejects booking within 2 hours
12. If accepted: booking status='ACCEPTED'
13. If rejected/timeout: auto-refund, booking status='CANCELLED'

### 4. Booking Lifecycle (State Machine)

### Strict State Transitions

PENDING -> ACCEPTED (provider action)
ACCEPTED -> IN_PROGRESS (provider starts service)
IN_PROGRESS -> COMPLETED (provider completes, client confirms)
COMPLETED -> (final state, can add review)

PENDING -> CANCELLED (client/provider/admin)
ACCEPTED -> CANCELLED (client/provider/admin, refund policy applies)

CANCELLED -> REFUNDED (admin approves refund)

### State Machine Validation (Backend)

javascript

```
const ALLOWED_TRANSITIONS = {
  PENDING: ['ACCEPTED', 'CANCELLED'],
  ACCEPTED: ['IN_PROGRESS', 'CANCELLED'],
  IN_PROGRESS: ['COMPLETED'],
  COMPLETED: [],
  CANCELLED: ['REFUNDED'],
  REFUNDED: []
};

function canTransition(currentStatus, newStatus) {
  return ALLOWED_TRANSITIONS[currentStatus]?.includes(newStatus);
}

// API endpoint example
app.patch('/api/bookings/:id/status', async (req, res) => {
  const { newStatus } = req.body;
  const booking = await getBooking(req.params.id);

  if (!canTransition(booking.status, newStatus)) {
    return res.status(400).json({ error: 'Invalid status transition' });
  }

  // Update booking status
  await updateBookingStatus(booking.id, newStatus);

  // Send notifications
  await sendStatusChangeNotification(booking);

  res.json({ success: true });
});
```

### 5. Review & Rating System

#### Review Submission

1. Booking must be in 'COMPLETED' status
2. Client can review within 30 days of completion
3. Client submits:
   - Rating (1-5 stars)
   - Written comment (optional)
   - Images (optional, max 3)
4. Create review record (is_verified=false)
5. Admin can flag inappropriate reviews
6. Calculate new average rating for service/provider
7. Update service.average_rating
8. Provider can respond to review
9. Review appears on service detail page

## 6. Dispute Resolution

### Dispute Flow

1. Either party (client/provider) raises dispute
2. Select dispute type:
   - Payment issue
   - Service quality
   - No-show
   - Cancellation disagreement
   - Other
3. Upload evidence (photos, messages, receipts)
4. Admin receives dispute notification
5. Admin reviews evidence from both sides
6. Admin can:
   - Request more information
   - Mediate conversation
   - Make final decision
7. Admin provides resolution:
   - Refund to client
   - Payment release to provider
   - Partial refund
   - No action
8. Update dispute status='RESOLVED'
9. Log resolution in admin_logs
10. Notify both parties

---

## 🔐 SECURITY IMPLEMENTATION (BANK-GRADE)

### Authentication Security

javascript

```
// JWT Configuration
{
  accessToken: {
    secret: process.env.JWT_ACCESS_SECRET, // 512-bit random
    expiresIn: '15m',
    algorithm: 'HS512'
  },
  refreshToken: {
    secret: process.env.JWT_REFRESH_SECRET, // Different secret
    expiresIn: '7d',
    algorithm: 'HS512'
  }
}

// Password Hashing
bcrypt.hash(password, 12); // 12 rounds

// Brute Force Protection
const rateLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 failed attempts
  message: 'Too many login attempts, please try after 15 minutes',
  standardHeaders: true,
  legacyHeaders: false,
  store: new RedisStore({ client: redis })
});

app.post('/api/auth/login', rateLimiter, loginHandler);
```

### API Rate Limiting

javascript

```
// General API rate limit
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100, // 100 requests per 15 min
  standardHeaders: true,
  store: new RedisStore({ client: redis })
});

// Booking creation limit
const bookingLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 10, // 10 bookings per hour
  message: 'Maximum 10 bookings per hour allowed',
  store: new RedisStore({ client: redis })
});

app.post('/api/bookings', authenticateJWT, bookingLimiter, createBooking);
```

### Input Validation (Zod)

typescript

```
// Booking validation schema
const createBookingSchema = z.object({
  serviceId: z.string().uuid(),
  scheduledDate: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  scheduledTime: z.string().regex(/^([01]\d|2[0-3]):([0-5]\d)$/),
  address: z.string().min(10).max(500),
  latitude: z.number().min(-90).max(90),
  longitude: z.number().min(-180).max(180),
  specialInstructions: z.string().max(1000).optional()
});

app.post('/api/bookings', async (req, res) => {
  try {
    const data = createBookingSchema.parse(req.body);
    // Proceed with validated data
  } catch (error) {
    return res.status(400).json({ errors: error.errors });
  }
});
```

### SQL Injection Prevention

typescript

```
// Using Prisma (parameterized queries)
const booking = await prisma.booking.findFirst({
  where: {
    id: bookingId, // Automatically sanitized
    clientId: userId
  },
  include: {
    service: true,
    provider: true,
    payment: true
  }
});

// NEVER do this:
// const query = `SELECT * FROM bookings WHERE id = '${bookingId}'`; // ❌ VULNERABLE
```

### XSS Protection

```
javascript

// Helmet middleware
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", "'unsafe-inline'", "https://maps.googleapis.com"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:", "https://*.cloudfront.net"],
      connectSrc: ["'self'", "https://api.bookyourservice.co.in"],
      fontSrc: ["'self'"],
      objectSrc: ["'none'"],
      mediaSrc: ["'self'"],
      frameSrc: ["'none'"]
    }
  },
  xssFilter: true,
  noSniff: true,
  referrerPolicy: { policy: 'strict-origin-when-cross-origin' }
}));

// DOMPurify for frontend
import DOMPurify from 'dompurify';
const cleanHTML = DOMPurify.sanitize(userInput);
```

### CSRF Protection

```javascript
import csrf from 'csurf';

const csrfProtection = csrf({
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict'
  }
});

app.use(csrfProtection);

app.get('/api/csrf-token', (req, res) => {
  res.json({ csrfToken: req.csrfToken() });
});

// Frontend must include token in POST requests
fetch('/api/bookings', {
  method: 'POST',
  headers: {
    'CSRF-Token': csrfToken
  },
  body: JSON.stringify(data)
});
```

### Payment Security

```javascript
```

```
// Razorpay webhook verification
app.post('/api/payments/webhook', (req, res) => {
  const signature = req.headers['x-razorpay-signature'];
  const body = JSON.stringify(req.body);

  const expectedSignature = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET)
    .update(body)
    .digest('hex');

  if (signature !== expectedSignature) {
    return res.status(400).json({ error: 'Invalid signature' });
  }

  // Process webhook
  const { event, payload } = req.body;

  if (event === 'payment.captured') {
    await markPaymentSuccess(payload.payment.entity.id);
  }

  res.json({ success: true });
});
```

### Data Encryption

```
javascript
```

```
// Encrypt sensitive data at rest (AES-256)
const crypto = require('crypto');

function encrypt(text) {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv(
    'aes-256-gcm',
    Buffer.from(process.env.ENCRYPTION_KEY, 'hex'),
    iv
  );

  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');

  const authTag = cipher.getAuthTag();

  return {
    iv: iv.toString('hex'),
    encryptedData: encrypted,
    authTag: authTag.toString('hex')
  };
}

// Store encrypted KYC document numbers
const encryptedAadhaar = encrypt(aadhaarNumber);
await prisma.providerKyc.create({
  data: {
    providerId,
    documentType: 'AADHAAR',
    documentNumber: JSON.stringify(encryptedAadhaar)
  }
});
```

---

## 🌍 GOOGLE MAPS INTEGRATION

### 1. Location Autocomplete

```
typescript
```

```
// Frontend component
import { Autocomplete } from '@react-google-maps/api';

function AddressInput({ onSelect }) {
  const [autocomplete, setAutocomplete] = useState(null);

  const onLoad = (autocompleteInstance) => {
    setAutocomplete(autocompleteInstance);
  };

  const onPlaceChanged = () => {
    if (autocomplete) {
      const place = autocomplete.getPlace();
      onSelect({
        address: place.formatted_address,
        latitude: place.geometry.location.lat(),
        longitude: place.geometry.location.lng(),
        city: place.address_components.find(c => c.types.includes('locality'))?.long_name,
        state: place.address_components.find(c => c.types.includes('administrative_area_level_1'))?.long_name,
        country: place.address_components.find(c => c.types.includes('country'))?.long_name,
        pincode: place.address_components.find(c => c.types.includes('postal_code'))?.long_name
      });
    }
  };

  return (
    <Autocomplete onLoad={onLoad} onPlaceChanged={onPlaceChanged}>
      <input
        type="text"
        placeholder="Enter your location"
        className="w-full p-2 border rounded"
      />
    </Autocomplete>
  );
}
```

## 2. Distance Calculation

javascript

```
// Backend distance calculation
const { Client } = require('@googlemaps/google-maps-services-js');
const client = new Client({});

async function calculateDistance(origin, destination) {
  const response = await client.distancematrix({
    params: {
      origins: [{ lat: origin.lat, lng: origin.lng }],
      destinations: [{ lat: destination.lat, lng: destination.lng }],
      mode: 'driving',
      key: process.env.GOOGLE_MAPS_API_KEY
    }
  });

  const element = response.data.rows[0].elements[0];

  return {
    distance: element.distance.value / 1000, // in km
    duration: element.duration.value / 60, // in minutes
    distanceText: element.distance.text,
    durationText: element.duration.text
  };
}

// Use in booking creation
app.post('/api/bookings', async (req, res) => {
  const { serviceId, serviceAddress } = req.body;
  const service = await getService(serviceId);

  const distance = await calculateDistance(
    { lat: service.latitude, lng: service.longitude },
    { lat: serviceAddress.latitude, lng: serviceAddress.longitude }
  );

  // Check if within service area
  if (distance.distance > service.serviceAreaRadiusKm) {
    return res.status(400).json({ error: 'Service not available in your area' });
  }

  // Calculate final price (distance surcharge if applicable)
  const finalPrice = service.basePrice + (distance.distance * 10); // ₹10 per km

  // Create booking...
});
```

### 3. Service Provider Map View

typescript

```
// Frontend map component
import { GoogleMap, Marker, InfoWindow } from '@react-google-maps/api';

function ServiceMap({ services, center }) {
  const [selected, setSelected] = useState(null);

  return (
    <GoogleMap
      zoom={12}
      center={center}
      mapContainerStyle={{ width: '100%', height: '500px' }}
    >
      {services.map(service => (
        <Marker
          key={service.id}
          position={{ lat: service.latitude, lng: service.longitude }}
          onClick={() => setSelected(service)}
          icon={{
            url: '/marker-icon.png',
            scaledSize: new window.google.maps.Size(40, 40)
          }}
        />
      ))}

      {selected && (
        <InfoWindow
          position={{ lat: selected.latitude, lng: selected.longitude }}
          onCloseClick={() => setSelected(null)}
        >
          <div className="p-2">
            <h3 className="font-bold">{selected.title}</h3>
            <p className="text-sm">{selected.provider.name}</p>
            <p className="text-sm">⭐ {selected.averageRating} ({selected.totalReviews})</p>
            <p className="text-green-600 font-bold">₹{selected.basePrice}</p>
            <button className="mt-2 bg-blue-600 text-white px-4 py-1 rounded">
              View Details
            </button>
          </div>
        </InfoWindow>
      )}
    </GoogleMap>
  );
}
```

## 📱 COMPLETE PAGE STRUCTURE

### Public Pages

1. **Landing Page** (`/`)
    - Hero section with search bar
    - Popular categories grid (25+ categories)
    - How it works (3-step process)
    - Featured services carousel
    - Testimonials
    - Stats (total bookings, providers, reviews)
    - CTA buttons (Sign Up as Provider, Book a Service)
2. **Categories Page** (`/categories`)
    - All 25+ categories with icons
    - Subcategories on hover/click (15+ each)
    - Filter by location, price range
    - Sort by popularity, newest
3. **Category Detail** (`/categories/:slug`)
    - Category description
    - All subcategories
    - Popular services in category
    - Filter/sort options
4. **Service Detail** (`/services/:id`)
    - Service images gallery
    - Title, description, pricing
    - Provider profile (name, rating, experience)
    - Location on map
    - Availability calendar
    - Reviews & ratings
    - Similar services
    - Book Now button
5. **Search Results** (`/search?q=...&location=...`)
    - Search filters (category, price, rating, distance)
    - Map view toggle
    - List of matching services

- Pagination

6. **About Us** (`/about`)
    - Company mission, vision
    - Team information
    - Milestones
7. **How It Works** (`/how-it-works`)
    - For clients: step-by-step booking process
    - For providers: signup to earnings process
    - Video tutorials
8. **FAQ** (`/faq`)
    - Categorized questions (General, Booking, Payment, Provider)
    - Search functionality
    - Contact support CTA
9. **Contact Us** (`/contact`)
    - Contact form
    - Email, phone, address
    - Office location on map
    - Social media links
10. **Blog** (`/blog`) (Optional)
    - Service tips, maintenance guides
    - Provider success stories
    - Platform updates
11. **Legal Pages**
    - Terms & Conditions (`/terms`)
    - Privacy Policy (`/privacy`)
    - Refund Policy (`/refund-policy`)
    - Cookie Policy (`/cookies`)

**Authentication Pages**

12. **Login** (`/login`)
    - Email/phone + password
    - Social login (Google, Facebook)
    - Remember me checkbox
    - Forgot password link

- Signup link

13. **Register** (`/register`)
    - Role selection (Client/Provider)
    - Email, phone, password, name
    - OTP verification
    - Terms acceptance checkbox
14. **Forgot Password** (`/forgot-password`)
    - Email/phone input
    - OTP verification
    - New password form
15. **Email Verification** (`/verify-email/:token`)
    - Success/failure message
    - Resend link

**Client Dashboard**

16. **Client Dashboard** (`/client/dashboard`)
    - Upcoming bookings
    - Past bookings
    - Favorite providers
    - Quick book again
    - Notifications
17. **Client Bookings** (`/client/bookings`)
    - All bookings (tabs: Upcoming, In Progress, Completed, Cancelled)
    - Filter by date, service, status
    - Booking actions (view, cancel, reschedule)
18. **Booking Detail** (`/client/bookings/:id`)
    - Full booking information
    - Provider contact
    - Service address on map
    - Payment receipt
    - Actions: Cancel, Contact Provider, Leave Review
19. **Client Profile** (`/client/profile`)
    - Personal information
    - Saved addresses

- Payment methods
- Change password
- Delete account

20. **Client Notifications** (`/client/notifications`)
    - All notifications (read/unread)
    - Mark as read, delete
21. **Client Reviews** (`/client/reviews`)
    - Reviews given by client
    - Edit/delete review
22. **Client Favorites** (`/client/favorites`)
    - Saved services
    - Favorite providers

**Provider Dashboard**

23. **Provider Dashboard** (`/provider/dashboard`)
    - Earnings summary (today, week, month)
    - New booking requests
    - Today's schedule
    - Rating & reviews stats
    - Quick actions (create service, view bookings)
24. **Provider Services** (`/provider/services`)
    - All services (active, pending approval, rejected)
    - Create new service
    - Edit/delete service
25. **Create/Edit Service** (`/provider/services/create`)
    - Form with category, subcategory, title, description
    - Pricing, negotiation toggle
    - Availability schedule
    - Service area selection (map)
    - Image upload
26. **Provider Bookings** (`/provider/bookings`)
    - Tabs: New Requests, Accepted, In Progress, Completed, Cancelled
    - Accept/reject new bookings
    - Start/complete service

27. **Booking Detail** (`/provider/bookings/:id`)
    - Client information
    - Service details
    - Location on map
    - Actions: Accept, Reject, Start, Complete, Contact Client
28. **Provider Earnings** (`/provider/earnings`)
    - Total earnings, pending payout, paid
    - Transaction history
    - Downloadable statements
    - Payout settings
29. **Provider Reviews** (`/provider/reviews`)
    - All reviews received
    - Average rating
    - Respond to reviews
30. **Provider Profile** (`/provider/profile`)
    - Business information
    - KYC documents (view status)
    - Service areas
    - Bank account details
    - Change password
31. **Provider KYC** (`/provider/kyc`)
    - Upload documents (ID, selfie, business license)
    - Verification status
    - Rejection reason (if any)
    - Resubmit documents

**Admin Dashboard**

32. **Admin Dashboard** (`/admin/dashboard`)
    - Key metrics (users, bookings, revenue, active disputes)
    - Charts (daily bookings, revenue trends)
    - Recent activities
    - Quick actions
33. **User Management** (`/admin/users`)
    - All users (tabs: All, Clients, Providers, Admins)
    - Search, filter, sort

- View, edit, block, delete user

34. **User Detail** (`/admin/users/:id`)
    - Full user profile
    - Activity history
    - Actions: Block, Unblock, Delete, Send Message
35. **Provider Verification** (`/admin/providers/verification`)
    - Pending KYC verifications
    - View documents
    - Approve/reject with reason
36. **Service Management** (`/admin/services`)
    - All services (pending approval, active, rejected)
    - Approve/reject services
    - Edit service details
    - Suspend service
37. **Booking Management** (`/admin/bookings`)
    - All bookings
    - Filter by status, date, service
    - View booking details
    - Override booking status
    - Issue refunds
38. **Payment Management** (`/admin/payments`)
    - All transactions
    - Failed payments
    - Refund requests
    - Manual refund processing
39. **Dispute Management** (`/admin/disputes`)
    - All disputes (open, under review, resolved)
    - View evidence from both parties
    - Mediate, provide resolution
    - Close dispute
40. **Category Management** (`/admin/categories`)
    - Add, edit, delete categories
    - Manage subcategories
    - Reorder display

41. **FAQ Management** (`/admin/faq`)
    - Add, edit, delete FAQs
    - Categorize questions
    - Reorder display
42. **Legal Pages** (`/admin/legal`)
    - Edit Terms, Privacy, Refund policies
    - Version control
43. **SEO Management** (`/admin/seo`)
    - Meta titles, descriptions for pages
    - Keyword management
    - Schema markup
    - Sitemap generation
44. **Analytics** (`/admin/analytics`)
    - User acquisition sources
    - Booking conversion funnel
    - Revenue analytics
    - Provider performance
    - Geographic heatmaps
45. **Reports** (`/admin/reports`)
    - Revenue reports
    - User growth reports
    - Service performance reports
    - Export to PDF/Excel
46. **Settings** (`/admin/settings`)
    - Platform fee configuration
    - Email templates
    - Notification settings
    - Payment gateway settings
    - Google Maps API key
47. **Admin Logs** (`/admin/logs`)
    - All admin actions
    - Filter by admin, action type, date
    - Audit trail

---

## 🔗 COMPLETE API ENDPOINTS

### Authentication

- `POST /api/auth/register` - User registration
- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `POST /api/auth/refresh` - Refresh access token
- `POST /api/auth/forgot-password` - Request password reset
- `POST /api/auth/reset-password` - Reset password with token
- `POST /api/auth/verify-email` - Verify email with OTP/token
- `POST /api/auth/resend-verification` - Resend verification email

### Users

- `GET /api/users/profile` - Get current user profile
- `PATCH /api/users/profile` - Update profile
- `PATCH /api/users/password` - Change password
- `DELETE /api/users/account` - Delete account
- `GET /api/users/:id` - Get user by ID (admin)
- `PATCH /api/users/:id/status` - Update user status (admin)

### Categories

- `GET /api/categories` - Get all categories
- `GET /api/categories/:id` - Get category by ID
- `GET /api/categories/:id/subcategories` - Get subcategories
- `POST /api/categories` - Create category (admin)
- `PATCH /api/categories/:id` - Update category (admin)
- `DELETE /api/categories/:id` - Delete category (admin)

### Services

- `GET /api/services` - Search/filter services
- `GET /api/services/:id` - Get service detail
- `POST /api/services` - Create service (provider)
- `PATCH /api/services/:id` - Update service (provider)
- `DELETE /api/services/:id` - Delete service (provider)
- `GET /api/services/:id/availability` - Get available slots

- `GET /api/services/:id/reviews` - Get service reviews
- `PATCH /api/services/:id/approve` - Approve service (admin)
- `PATCH /api/services/:id/reject` - Reject service (admin)

**Bookings**

- `GET /api/bookings` - Get user bookings
- `GET /api/bookings/:id` - Get booking detail
- `POST /api/bookings` - Create booking
- `PATCH /api/bookings/:id/accept` - Accept booking (provider)
- `PATCH /api/bookings/:id/reject` - Reject booking (provider)
- `PATCH /api/bookings/:id/start` - Start service (provider)
- `PATCH /api/bookings/:id/complete` - Complete service (provider)
- `PATCH /api/bookings/:id/cancel` - Cancel booking
- `POST /api/bookings/:id/negotiate` - Propose price negotiation

**Payments**

- `POST /api/payments/create-order` - Create payment order
- `POST /api/payments/verify` - Verify payment (client-side)
- `POST /api/payments/webhook` - Payment gateway webhook
- `GET /api/payments/:id` - Get payment details
- `POST /api/payments/:id/refund` - Process refund (admin)

**Reviews**

- `POST /api/reviews` - Submit review
- `GET /api/reviews/:id` - Get review detail
- `PATCH /api/reviews/:id` - Update review
- `DELETE /api/reviews/:id` - Delete review
- `POST /api/reviews/:id/response` - Provider response to review

**Disputes**

- `POST /api/disputes` - Raise dispute
- `GET /api/disputes` - Get user disputes
- `GET /api/disputes/:id` - Get dispute detail
- `PATCH /api/disputes/:id/resolve` - Resolve dispute (admin)
- `POST /api/disputes/:id/messages` - Add message to dispute

**KYC**

- `POST /api/kyc/submit` - Submit KYC documents
- `GET /api/kyc/status` - Get KYC status
- `PATCH /api/kyc/:id/approve` - Approve KYC (admin)
- `PATCH /api/kyc/:id/reject` - Reject KYC (admin)

**Notifications**

- `GET /api/notifications` - Get user notifications
- `PATCH /api/notifications/:id/read` - Mark as read
- `DELETE /api/notifications/:id` - Delete notification

**Admin**

- `GET /api/admin/dashboard` - Dashboard stats
- `GET /api/admin/users` - List all users
- `GET /api/admin/analytics` - Analytics data
- `GET /api/admin/logs` - Admin activity logs

**FAQ**

- `GET /api/faq` - Get all FAQs
- `POST /api/faq` - Create FAQ (admin)
- `PATCH /api/faq/:id` - Update FAQ (admin)
- `DELETE /api/faq/:id` - Delete FAQ (admin)

**SEO**

- `GET /api/seo/:pageType/:pageId` - Get SEO metadata
- `PATCH /api/seo/:pageType/:pageId` - Update SEO metadata (admin)

---

## 🎨 FRONTEND IMPLEMENTATION DETAILS

**Tech Stack**

- React 18 + TypeScript
- Vite (build tool)
- React Router 6 (routing)
- React Query (server state management)

- Context API (global state)
- Tailwind CSS (styling)
- shadcn/ui (UI components)
- React Hook Form (forms)
- Zod (validation)
- Axios (HTTP client)
- React Google Maps API (maps)
- Framer Motion (animations)
- Recharts (charts for admin)
- React Hot Toast (notifications)
- React Helmet (SEO)

**Project Structure**

```
src/
├── components/
│   ├── common/
│   │   ├── Button.tsx
│   │   ├── Input.tsx
│   │   ├── Modal.tsx
│   │   ├── Spinner.tsx
│   │   └── ...
│   ├── layout/
│   │   ├── Header.tsx
│   │   ├── Footer.tsx
│   │   ├── Sidebar.tsx
│   │   └── Layout.tsx
│   ├── auth/
│   │   ├── LoginForm.tsx
│   │   ├── RegisterForm.tsx
│   │   └── ...
│   ├── service/
│   │   ├── ServiceCard.tsx
│   │   ├── ServiceFilter.tsx
│   │   ├── ServiceMap.tsx
│   │   └── ...
│   ├── booking/
│   │   ├── BookingCard.tsx
│   │   ├── BookingForm.tsx
│   │   └── ...
│   └── ...
├── pages/
│   ├── public/
```

```
│   │   ├── Home.tsx
│   │   ├── Categories.tsx
│   │   ├── ServiceDetail.tsx
│   │   └── ...
│   ├── client/
│   │   ├── Dashboard.tsx
│   │   ├── Bookings.tsx
│   │   └── ...
│   ├── provider/
│   │   ├── Dashboard.tsx
│   │   ├── Services.tsx
│   │   └── ...
│   └── admin/
│       ├── Dashboard.tsx
│       ├── Users.tsx
│       └── ...
├── context/
│   ├── AuthContext.tsx
│   ├── NotificationContext.tsx
│   └── ...
├── hooks/
│   ├── useAuth.ts
│   ├── useBookings.ts
│   ├── useServices.ts
│   └── ...
├── services/
│   ├── api.ts
│   ├── auth.service.ts
│   ├── booking.service.ts
│   ├── service.service.ts
│   └── ...
├── utils/
│   ├── validation.ts
│   ├── formatters.ts
│   ├── constants.ts
│   └── ...
├── types/
│   ├── user.ts
│   ├── service.ts
│   ├── booking.ts
│   └── ...
├── styles/
│   └── globals.css
├── App.tsx
└── main.tsx
```

**Responsive Design**

- Mobile-first approach
- Breakpoints: sm (640px), md (768px), lg (1024px), xl (1280px)
- Touch-friendly UI (min 44px touch targets)
- PWA support (installable on mobile)

**Performance Optimization**

- Code splitting (React.lazy, Suspense)
- Image optimization (lazy loading, WebP format)
- Virtual scrolling for long lists
- Debouncing search inputs
- Memoization (React.memo, useMemo, useCallback)
- Service Worker for offline support

**SEO Implementation**

- React Helmet for dynamic meta tags
- Server-side rendering (optional: Next.js migration)
- Sitemap generation
- Robots.txt
- Structured data (JSON-LD schema)
- Canonical URLs
- Open Graph tags
- Twitter Cards

---

## 🚀 DEPLOYMENT & DEVOPS

**Development Environment**

```bash
```

```
# Backend
cd backend
cp .env.example .env
npm install
npx prisma generate
npx prisma migrate dev
npm run dev

# Frontend
cd frontend
cp .env.example .env
npm install
npm run dev
```

### Docker Compose (Local/Staging)

yaml

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: bookyourservice
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: bookyourservice_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bookyourservice"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    environment:
      DATABASE_URL: postgresql://bookyourservice:${DB_PASSWORD}@postgres:5432/bookyourservice_db
      REDIS_URL: redis://redis:6379
      JWT_ACCESS_SECRET: ${JWT_ACCESS_SECRET}
      JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET}
    ports:
      - "3000:3000"
    depends_on:
      - postgres
      - redis
    volumes:
      - ./backend:/app
      - /app/node_modules

  frontend:
    build:
```

```yaml
    context: ./frontend
    dockerfile: Dockerfile
  environment:
    VITE_API_URL: http://localhost:3000
    VITE_GOOGLE_MAPS_API_KEY: ${GOOGLE_MAPS_API_KEY}
  ports:
    - "5173:80"
  depends_on:
    - backend

nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    - ./nginx/ssl:/etc/nginx/ssl
  depends_on:
    - frontend
    - backend

volumes:
postgres_data:
redis_data:
```

### Kubernetes Deployment (Production)

yaml

```yaml
# backend-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: bookyourservice
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: bookyourservice/backend:latest
        ports:
        - containerPort: 3000
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: database-url
        - name: REDIS_URL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: redis-url
        resources:
          requests:
            memory: "512Mi"
            cpu: "500m"
          limits:
            memory: "1Gi"
            cpu: "1000m"
        livenessProbe:
          httpGet:
            path: /health/live
            port: 3000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
```

```yaml
        httpGet:
          path: /health/ready
          port: 3000
        initialDelaySeconds: 5
        periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: bookyourservice
spec:
  selector:
    app: backend
  ports:
  - protocol: TCP
    port: 80
    targetPort: 3000
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: bookyourservice
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
```

### CI/CD Pipeline (GitHub Actions)

yaml

```yaml
# .github/workflows/deploy.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test_db
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
      redis:
        image: redis:7
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'

      - name: Install Backend Dependencies
        run: |
          cd backend
          npm ci

      - name: Run Backend Tests
        run: |
```

```
        cd backend
        npm run test
      env:
        DATABASE_URL: postgresql://test:test@localhost:5432/test_db
        REDIS_URL: redis://localhost:6379

    - name: Install Frontend Dependencies
      run: |
        cd frontend
        npm ci

    - name: Run Frontend Tests
      run: |
        cd frontend
        npm run test

  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
    - uses: actions/checkout@v3

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v2

    - name: Login to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Build and Push Backend
      uses: docker/build-push-action@v4
      with:
        context: ./backend
        push: true
        tags: bookyourservice/backend:latest

    - name: Build and Push Frontend
      uses: docker/build-push-action@v4
      with:
        context: ./frontend
        push: true
        tags: bookyourservice/frontend:latest
```

```
deploy:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v3

    - name: Configure AWS Credentials
      uses: aws-actions/configure-aws-credentials@v2
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ap-south-1

    - name: Deploy to EKS
      run: |
        aws eks update-kubeconfig --name bookyourservice-cluster
        kubectl apply -f k8s/
        kubectl rollout restart deployment/backend -n bookyourservice
        kubectl rollout restart deployment/frontend -n bookyourservice
```

### Monitoring & Logging

javascript

```
// Winston logger configuration
const winston = require('winston');
const { ElasticsearchTransport } = require('winston-elasticsearch');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'bookyourservice-backend' },
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' }),
    new ElasticsearchTransport({
      level: 'info',
      clientOpts: { node: process.env.ELASTICSEARCH_URL },
      index: 'bookyourservice-logs'
    })
  ]
});

// Prometheus metrics
const promClient = require('prom-client');
const register = new promClient.Registry();

promClient.collectDefaultMetrics({ register });

const httpRequestDuration = new promClient.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  registers: [register]
});

// Middleware to track request duration
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = (Date.now() - start) / 1000;
    httpRequestDuration.labels(req.method, req.route?.path || req.path, res.statusCode).observe(duration);
  });
  next();
});
```

```
// Metrics endpoint
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});
```

### Backup Strategy

bash

```
#!/bin/bash
# scripts/backup-database.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/backups/postgres"
BACKUP_FILE="$BACKUP_DIR/backup_$DATE.sql.gz"

# Create backup
pg_dump -U bookyourservice -h postgres bookyourservice_db | gzip > $BACKUP_FILE

# Upload to S3
aws s3 cp $BACKUP_FILE s3://bookyourservice-backups/database/

# Delete backups older than 30 days
find $BACKUP_DIR -type f -name "backup_*.sql.gz" -mtime +30 -delete

# Cron: 0 2 * * * /scripts/backup-database.sh
```

---

## 📈 SEO & MARKETING

### Google Search Console Setup

1. Verify domain ownership
2. Submit sitemap.xml
3. Monitor indexing status
4. Fix crawl errors

### Sitemap Generation

typescript

```ts
// scripts/generate-sitemap.ts
import { SitemapStream, streamToPromise } from 'sitemap';
import { createWriteStream } from 'fs';
import prisma from './prisma';

async function generateSitemap() {
  const sitemap = new SitemapStream({ hostname: 'https://bookyourservice.co.in' });
  const writeStream = createWriteStream('./public/sitemap.xml');

  sitemap.pipe(writeStream);

  // Static pages
  sitemap.write({ url: '/', changefreq: 'daily', priority: 1.0 });
  sitemap.write({ url: '/about', changefreq: 'monthly', priority: 0.7 });
  sitemap.write({ url: '/how-it-works', changefreq: 'monthly', priority: 0.7 });
  sitemap.write({ url: '/faq', changefreq: 'weekly', priority: 0.6 });

  // Categories
  const categories = await prisma.serviceCategory.findMany({ where: { isActive: true } });
  categories.forEach(cat => {
    sitemap.write({ url: `/categories/${cat.slug}`, changefreq: 'weekly', priority: 0.8 });
  });

  // Services
  const services = await prisma.service.findMany({
    where: { isActive: true, isApproved: true },
    select: { id: true, updatedAt: true }
  });
  services.forEach(service => {
    sitemap.write({
      url: `/services/${service.id}`,
      lastmod: service.updatedAt.toISOString(),
      changefreq: 'weekly',
      priority: 0.9
    });
  });

  sitemap.end();
  await streamToPromise(sitemap);
  console.log('Sitemap generated successfully');
}

generateSitemap();
// Run daily via cron
```

### Schema Markup (JSON-LD)

typescript

```
// components/ServiceDetail.tsx
function ServiceDetail({ service }) {
  const schemaMarkup = {
    "@context": "https://schema.org",
    "@type": "Service",
    "name": service.title,
    "description": service.description,
    "provider": {
      "@type": "LocalBusiness",
      "name": service.provider.name,
      "aggregateRating": {
        "@type": "AggregateRating",
        "ratingValue": service.averageRating,
        "reviewCount": service.totalReviews
      }
    },
    "offers": {
      "@type": "Offer",
      "price": service.basePrice,
      "priceCurrency": "INR"
    },
    "areaServed": {
      "@type": "GeoCircle",
      "geoMidpoint": {
        "@type": "GeoCoordinates",
        "latitude": service.latitude,
        "longitude": service.longitude
      },
      "geoRadius": `${service.serviceAreaRadiusKm} km`
    }
  };

  return (
    <>
      <Helmet>
        <script type="application/ld+json">
          {JSON.stringify(schemaMarkup)}
        </script>
      </Helmet>
      {/* Service detail UI */}
    </>
  );
}
```

## Google AdSense Integration

html

```
<!-- public/index.html -->
<head>
  <!-- AdSense verification -->
  <meta name="google-adsense-account" content="ca-pub-XXXXXXXXXXXXXXXX">

  <!-- Auto ads -->
  <script async src="https://pagead2.googlesyndication.com/pagead/js/adsbygoogle.js?client=ca-pub-XXXXXXXXXXXXXX
          crossorigin="anonymous"></script>
</head>

<!-- In-feed ad unit (React component) -->
<ins class="adsbygoogle"
     style="display:block"
     data-ad-format="fluid"
     data-ad-layout-key="-fb+5w+4e-db+86"
     data-ad-client="ca-pub-XXXXXXXXXXXXXXXX"
     data-ad-slot="XXXXXXXXXX"></ins>
<script>
     (adsbygoogle = window.adsbygoogle || []).push({});
</script>
```

## Analytics Integration

typescript

```
// Google Analytics 4
import ReactGA from 'react-ga4';

ReactGA.initialize('G-XXXXXXXXXX');

// Track page views
function usePageTracking() {
  const location = useLocation();

  useEffect(() => {
    ReactGA.send({ hitType: 'pageview', page: location.pathname });
  }, [location]);
}

// Track events
function trackBookingCreated(bookingId, serviceId, amount) {
  ReactGA.event({
    category: 'Booking',
    action: 'Created',
    label: serviceId,
    value: amount
  });
}

// Facebook Pixel
import ReactPixel from 'react-facebook-pixel';

ReactPixel.init('XXXXXXXXXXXXXXX');
ReactPixel.track('ViewContent', { content_name: 'Service Detail Page' });
```

---

## 🌐 INTERNATIONALIZATION (I18N)

### Multi-Language Support

typescript

```
// i18n configuration
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';

i18n
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    resources: {
      en: {
        translation: {
          "welcome": "Welcome to BookYourService",
          "search_services": "Search for services",
          "book_now": "Book Now"
        }
      },
      hi: {
        translation: {
          "welcome": "BookYourService में आपका स्वागत है",
          "search_services": "सेवाओं की खोज करें",
          "book_now": "अभी बुक करें"
        }
      }
    },
    fallbackLng: 'en',
    interpolation: {
      escapeValue: false
    }
  });

// Usage
import { useTranslation } from 'react-i18next';

function Home() {
  const { t, i18n } = useTranslation();

  return (
    <div>
      <h1>{t('welcome')}</h1>
      <button onClick={() => i18n.changeLanguage('hi')}>हिंदी</button>
    </div>
  );
}
```

### Multi-Currency Support

```typescript
// Currency configuration
const CURRENCIES = {
  IN: { code: 'INR', symbol: '₹' },
  US: { code: 'USD', symbol: '$' },
  GB: { code: 'GBP', symbol: '£' },
  EU: { code: 'EUR', symbol: '€' }
};

// Detect user country
async function detectUserCountry() {
  const response = await fetch('https://ipapi.co/json/');
  const data = await response.json();
  return data.country_code;
}

// Convert prices
async function convertPrice(amount, fromCurrency, toCurrency) {
  const response = await fetch(`https://api.exchangerate-api.com/v4/latest/${fromCurrency}`);
  const data = await response.json();
  return amount * data.rates[toCurrency];
}

// Display price
function PriceDisplay({ amount, currency }) {
  const { symbol } = CURRENCIES[currency];
  return <span>{symbol}{amount.toFixed(2)}</span>;
}
```

---

## ✅ LAUNCH CHECKLIST

### Pre-Launch (Development)

- [x] Database schema finalized and migrated
- [x] All API endpoints implemented and tested
- [x] Frontend pages completed (47 pages)
- [x] Authentication flow working (login, register, OTP)
- [x] Role-based access control enforced
- [x] KYC verification workflow implemented
- [x] Service creation and approval flow working
- [x] Booking state machine validated

- [ ] Payment gateway integrated and tested
- [ ] Price negotiation feature implemented
- [ ] Google Maps integration completed
- [ ] Review and rating system working
- [ ] Dispute resolution workflow implemented
- [ ] Notification system (email, SMS, push) working
- [ ] Admin dashboard functional
- [ ] SEO meta tags on all pages
- [ ] Sitemap generation automated
- [ ] Legal pages (Terms, Privacy, Refund) completed
- [ ] FAQ section populated

**Security Audit**

- [ ] All passwords hashed (bcrypt, 12 rounds)
- [ ] JWT tokens properly configured (15min access, 7day refresh)
- [ ] Rate limiting on all endpoints
- [ ] Input validation (Zod schemas)
- [ ] SQL injection prevention (Prisma ORM)
- [ ] XSS protection (DOMPurify, CSP headers)
- [ ] CSRF protection (csrf middleware)
- [ ] Payment webhook signature verification
- [ ] Sensitive data encrypted at rest (AES-256)
- [ ] HTTPS enforced (TLS 1.3)
- [ ] Security headers (Helmet.js)
- [ ] Dependency vulnerabilities scanned (npm audit)

**Performance Testing**

- [ ] Load testing (K6, 1000 concurrent users)
- [ ] Database query optimization (indexes added)
- [ ] Redis caching implemented
- [ ] CDN configured for static assets
- [ ] Image optimization (WebP, lazy loading)
- [ ] Code splitting (React.lazy)
- [ ] API response time < 500ms
- [ ] Page load time < 3 seconds
- [ ] Lighthouse score > 90

**DevOps Setup**

- [ ] Docker images created (multi-stage builds)
- [ ] Docker Compose for local development

- [x] Kubernetes manifests created
- [x] CI/CD pipeline configured (GitHub Actions)
- [x] Automated testing in CI
- [x] Database backup script created
- [x] Monitoring setup (Prometheus, Grafana)
- [x] Logging setup (Winston, ELK)
- [x] Error tracking (Sentry)
- [x] Uptime monitoring (UptimeRobot)

**Domain & Hosting**

- [ ] Domain purchased (bookyourservice.co.in)
- [ ] DNS configured
- [ ] SSL certificate obtained (Let's Encrypt)
- [ ] Server provisioned (AWS/GCP/Azure)
- [ ] Database hosted (RDS/Cloud SQL)
- [ ] Redis hosted (ElastiCache/Memorystore)
- [ ] S3/Cloud Storage for uploads
- [ ] CDN configured (CloudFront/Cloudflare)

**Third-Party Integrations**

- [ ] Payment gateway live credentials (Razorpay/Stripe)
- [ ] Google Maps API key (production quota)
- [ ] SMS gateway (Twilio/AWS SNS)
- [ ] Email service (SendGrid/AWS SES)
- [ ] Push notification service (FCM)
- [ ] Google Analytics tracking ID
- [ ] Facebook Pixel ID
- [ ] Google AdSense account approved

**Legal & Compliance**

- [ ] Privacy Policy reviewed by lawyer
- [ ] Terms & Conditions reviewed
- [ ] Refund Policy finalized
- [ ] GDPR compliance measures
- [ ] Cookie consent implemented
- [ ] Business registered
- [ ] Tax registration (GST)
- [ ] Payment gateway merchant account

**Marketing Ready**

- [ ] Google Search Console verified
- [ ] Sitemap submitted to Google
- [ ] Schema markup validated
- [ ] Social media pages created (Facebook, Instagram, Twitter)
- [ ] Blog ready (if applicable)
- [ ] Email templates designed
- [ ] Onboarding emails automated
- [ ] Referral program configured

**Launch Day**

- [ ] Deploy to production
- [ ] Smoke tests on production
- [ ] Monitor error logs
- [ ] Monitor server resources
- [ ] Check payment gateway webhooks
- [ ] Test user registration flow
- [ ] Test booking creation end-to-end
- [ ] Announce on social media
- [ ] Press release (if applicable)

**Post-Launch (Week 1)**

- [ ] Monitor user signups
- [ ] Monitor booking conversions
- [ ] Track payment success rate
- [ ] Review error reports
- [ ] Collect user feedback
- [ ] Fix critical bugs
- [ ] Optimize slow queries
- [ ] Adjust server resources

---

## 🎯 SUCCESS METRICS (KPIs)

**User Metrics**

- Total registered users (clients + providers)
- Daily/Monthly active users (DAU/MAU)
- User retention rate (Day 1, Day 7, Day 30)
- Average session duration

- Bounce rate

**Booking Metrics**

- Total bookings created
- Booking completion rate (% of bookings completed)
- Average booking value (ABV)
- Booking cancellation rate
- Provider acceptance rate
- Average time to provider acceptance

**Revenue Metrics**

- Gross Merchandise Value (GMV)
- Total revenue (platform fees + other streams)
- Revenue per booking
- Monthly Recurring Revenue (MRR) from subscriptions
- Average revenue per user (ARPU)

**Provider Metrics**

- Total providers registered
- KYC approval rate
- Active providers (% providing services)
- Average provider rating
- Provider earnings (total, average per provider)

**Quality Metrics**

- Average service rating
- Review submission rate
- Dispute rate (% of bookings)
- Dispute resolution time
- User satisfaction score (NPS)

**Technical Metrics**

- API uptime (target: 99.9%)
- Average API response time (target: <500ms)
- Error rate (target: <0.1%)

- Payment success rate (target: >95%)
- Page load time (target: <3s)

---

## 📚 DOCUMENTATION REQUIREMENTS

### Technical Documentation

1. **API Documentation** (Swagger/Postman)
    - All endpoints documented
    - Request/response examples
    - Authentication requirements
    - Error codes explained
2. **Database Documentation**
    - ER diagram
    - Table descriptions
    - Index strategy
    - Migration history
3. **Architecture Documentation**
    - System architecture diagram
    - Data flow diagrams
    - Security architecture
    - Deployment architecture
4. **Developer Guide**
    - Setup instructions
    - Code structure explanation
    - Contribution guidelines
    - Testing guide

### User Documentation

1. **User Guide (Clients)**
    - How to register
    - How to search for services
    - How to book a service
    - How to cancel/reschedule
    - How to leave a review

- Payment FAQs

2. **Provider Guide**
    - How to register as provider
    - KYC verification process
    - How to create a service
    - How to manage bookings
    - How to receive payments
    - Earnings and payouts
3. **Admin Guide**
    - User management
    - Service approval process
    - KYC verification
    - Dispute resolution
    - Platform settings
    - Analytics and reports

---

## 🚨 CRITICAL RULES (NEVER VIOLATE)

### Code Quality

- ✅ **TypeScript strict mode** everywhere
- ✅ **No any types** (use proper typing)
- ✅ **ESLint** and **Prettier** configured
- ✅ **100% test coverage** for critical paths (auth, payment, booking)
- ✅ **Code reviews mandatory** before merge

### Security

- ✅ **Never commit secrets** to git
- ✅ **Environment variables** for all configs
- ✅ **Validate all inputs** (backend validation required, frontend optional)
- ✅ **Sanitize all outputs** (prevent XSS)
- ✅ **Rate limit all endpoints**
- ✅ **Log all admin actions**

**Database**

- ✅ **Migrations only** (never manual schema changes)
- ✅ **Transactions** for multi-step operations
- ✅ **Indexes** on foreign keys and search fields
- ✅ **Soft delete** for critical tables
- ✅ **Daily backups** with 30-day retention

**Payments**

- ✅ **Webhook verification** mandatory
- ✅ **Idempotency keys** for payment operations
- ✅ **Refund approval** by admin only
- ✅ **Audit log** for all payment actions
- ✅ **Test mode** separate from production

**User Experience**

- ✅ **Mobile-first** design
- ✅ **Accessibility** (WCAG 2.1 AA)
- ✅ **Loading states** on all async operations
- ✅ **Error messages** user-friendly
- ✅ **Confirmation prompts** for destructive actions

---

## 🎬 FINAL MANDATE

Build BookYourService as a **world-class marketplace** that can compete with global platforms. Every line of code, every database query, every API endpoint must be **production-ready from day one**. No shortcuts, no "we'll fix it later", no technical debt.

This is not a prototype. This is not an MVP. This is a **full-scale, enterprise-ready platform** designed to handle millions of users, thousands of concurrent bookings, and billions in GMV.

**Quality over speed. Security over features. User trust above everything.**

Now, go build something extraordinary! 🚀